

Efficient Audio Preprocessing and TI NPU Architecture for Keyword Spotting

A MATLAB study of raw-audio preprocessing, compact feature tensors, and NPU-compatible DSCNN architecture for resource-constrained audio classification.

Student	Lucas(Wenqi) Wang
Student ID	550552222
GitHub username	wenqiwang1314-dotcom
Repository	https://github.com/wenqiwang1314-dotcom/elec5305-project-550552222
Project site	https://wenqiwang1314-dotcom.github.io/elec5305-project-550552222/

Project Overview

This project investigates an efficient audio-classification system that recognises ten spoken commands - down, go, left, no, off, on, right, stop, up, and yes - plus unknown speech and silence. Keyword spotting is the small-vocabulary front end used by voice-controlled devices before a larger speech recogniser is activated. It must be accurate enough to avoid false activations, but computationally small enough to run continuously. The research focus is the joint design of raw-audio conditioning, compact time-frequency features, integer tensor scaling, and an NPU-compatible depthwise-separable convolutional neural network (DSCNN). Texas Instruments' 12-class Google Speech Commands example for MSPM0G5187 is the reference architecture. The present evidence remains PC-side until compilation and embedded deployment are separately verified.

Background and Motivation

Always-on keyword spotting must balance recognition against preprocessing time, feature memory, model computation, and energy. A stronger frontend can reduce model burden but is not free. This project connects ELEC5305 framing, Fourier analysis, filterbanks, cepstral features, noise robustness, and objective evaluation to an end-to-end edge-audio design.

Proposed Methodology

The experiments will use Speech Commands v0.02, a public dataset containing 105,829 one-second spoken-word recordings at 16 kHz. The supplied official validation and testing lists will be respected so recordings from the same speaker do not leak across partitions. Ten commands will be retained as known classes. Other word folders will form the unknown class, and one-second regions from the background-noise recordings will form the silence class.

MATLAB first audits sample rate, channels, duration, clipping, and DC, then tests amplitude normalization, pre-emphasis, alignment, and noise augmentation. The TI front end - 30 ms frames, 20 ms step, 40 Mel filters, and 10 MFCCs - produces a 49 by 10 tensor. Controlled ablations compare frame/hop size, MFCC count, CMN,

log-Mel, and PCEN while recording feature cost. Promising variants will feed the same DSCNN matrix: the TI 64-filter/four-block anchor plus bounded width, depth, kernel, and stride reductions using NPU-supported operators. Float and int8 tensors will be checked with saved golden vectors before compilation.

Evaluation reports clean and noisy accuracy, macro/per-class recall, unknown rejection, calibration, false accepts per hour, and streaming latency. Frontend time/memory is separated from parameters, MACs, activation RAM, NPU coverage, and inference time. Model rankings require at least three seeds and validation-only selection. Hardware claims require compilation and measurement on the named target.

Expected Outcomes

Deliverables include MATLAB code, twelve downloadable class archives with 960 traceable WAVs, manifests and hashes, acoustic figures, metrics, a verified literature set, and GitHub Pages. The final target is a multi-seed DSCNN study that jointly reports preprocessing, recognition, robustness, and NPU cost.

Timeline (Weeks 1-13)

Weeks	Work package
1-2	Define the 12-class task, data contract, and literature baseline.
3-5	Test waveform conditioning, MFCC/MFSC/PCEN, and integer tensor scaling.
6-9	Compare NPU-compatible DSCNN width/depth designs with repeated seeds.
10-11	Compile and evaluate robustness, rejection, latency, memory, and energy.
12-13	Finalise the report, reproducibility package, site, and demonstration.

References

1. P. Warden, "Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition," arXiv:1804.03209, 2018.
2. Y. Zhang, N. Suda, L. Lai, and V. Chandra, "Hello Edge: Keyword Spotting on Microcontrollers," arXiv:1711.07128, 2017.
3. S. B. Davis and P. Mermelstein, "Comparison of Parametric Representations for Monosyllabic Word Recognition in Continuously Spoken Sentences," IEEE Transactions on Acoustics, Speech, and Signal Processing, vol. 28, no. 4, pp. 357-366, 1980, doi:10.1109/TASSP.1980.1163420.
4. T. N. Sainath and C. Parada, "Convolutional Neural Networks for Small-footprint Keyword Spotting," Proceedings of Interspeech, pp. 1478-1482, 2015, doi:10.21437/Interspeech.2015-147.
5. G. Tucker, M. Wu, M. Sun, S. Panchapagesan, G. Fu, and S. Vitaladevuni, "Model Compression Applied to Small-Footprint Keyword Spotting," Proceedings of Interspeech, pp. 1878-1882, 2016, doi:10.21437/Interspeech.2016-1393.
6. Texas Instruments, "Google Speech Command Recognition," Tiny ML Tensorlab 1.4.0 User Guide, 2026.
7. Y. Wang et al., "Trainable Frontend for Robust and Far-Field Keyword Spotting," ICASSP, 2017, doi:10.1109/ICASSP.2017.7953242.
8. P. M. Sorensen, B. Epp, and T. May, "A Depthwise Separable Convolutional Neural Network for Keyword Spotting on an Embedded System," EURASIP JASMP, 2020, doi:10.1186/s13636-020-00176-2.
9. D. Peter, W. Roth, and F. Pernkopf, "Resource-efficient DNNs for Keyword Spotting using Neural Architecture Search and Quantization," arXiv:2012.10138, 2020.
10. A. Osman et al., "TinyML Platforms Benchmarking," arXiv:2112.01319, 2021.
11. A. Almaini, J. Folz, and G. Ashour, "TinyML for Acoustic Anomaly Detection in IoT Sensor Networks," arXiv:2603.26135, 2026.

Preliminary Evidence

The first smoke test used 720 training and 240 held-out test utterances with a manual 5-nearest-neighbour classifier. It produced the required 49 by 10 MFCC map, found zero speaker overlap, and reached 39.17% accuracy and macro recall, above the 8.33% chance level. Silence recall was 100%, but unknown-word recall was 0%. On the identical manifest and classifier, adding utterance CMN raised clean accuracy to 47.08% at the same 490-element input; pre-emphasis reached 41.67%. Denser 98-frame inputs added little despite doubling or tripling the feature size. At 10 dB, every clean-trained variant fell to 8.33-10.42%, exposing noise mismatch as a more important next problem than increasing MFCC size. These are single-subset PC-side diagnostics, not DSCNN or embedded results.

MFCC Front-End Optimization

A controlled ablation keeps the 720/240 manifest and manual 5-NN fixed while changing only the acoustic front end. Clean-trained models are also tested with deterministic 10 dB additive noise. Feature elements are a transparent input-size proxy, not a target-RAM measurement.

Front end	Shape	Elements	Clean	10 dB
Tl reference	49 x 10	490	39.17%	8.33%
13 MFCC	49 x 13	637	40.42%	8.33%
10 ms hop	98 x 10	980	40.83%	8.33%
25/10 ms, 13 MFCC	98 x 13	1274	39.17%	8.33%
Tl + pre-emphasis	49 x 10	490	41.67%	8.33%
Tl + CMN	49 x 10	490	47.08%	10.00%
20-bin MFSC	49 x 20	980	28.75%	10.42%

CMN reaches 47.08% clean accuracy at the same 490-element input. The near-chance 10 dB results show that noise-aware training is a higher priority than increasing the feature map.

Appendix: Preliminary Real-Data Results

The following figures were generated by the submitted MATLAB smoke test from held-out Speech Commands v0.02 samples. The result is a 5-NN baseline, not a DSCNN or embedded-device measurement.

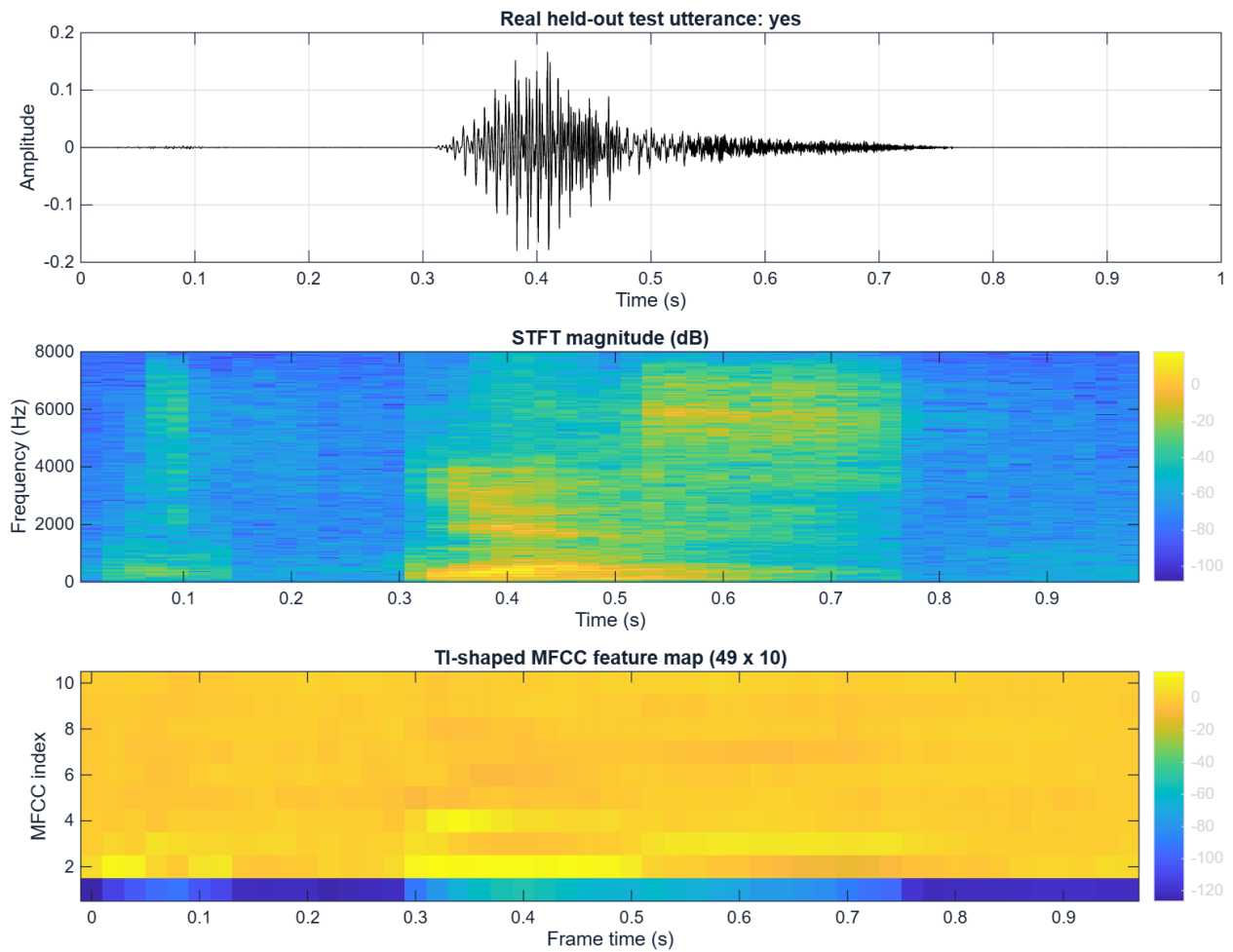


Figure A1. Real waveform, STFT, and TI-shaped 49 x 10 MFCC map.

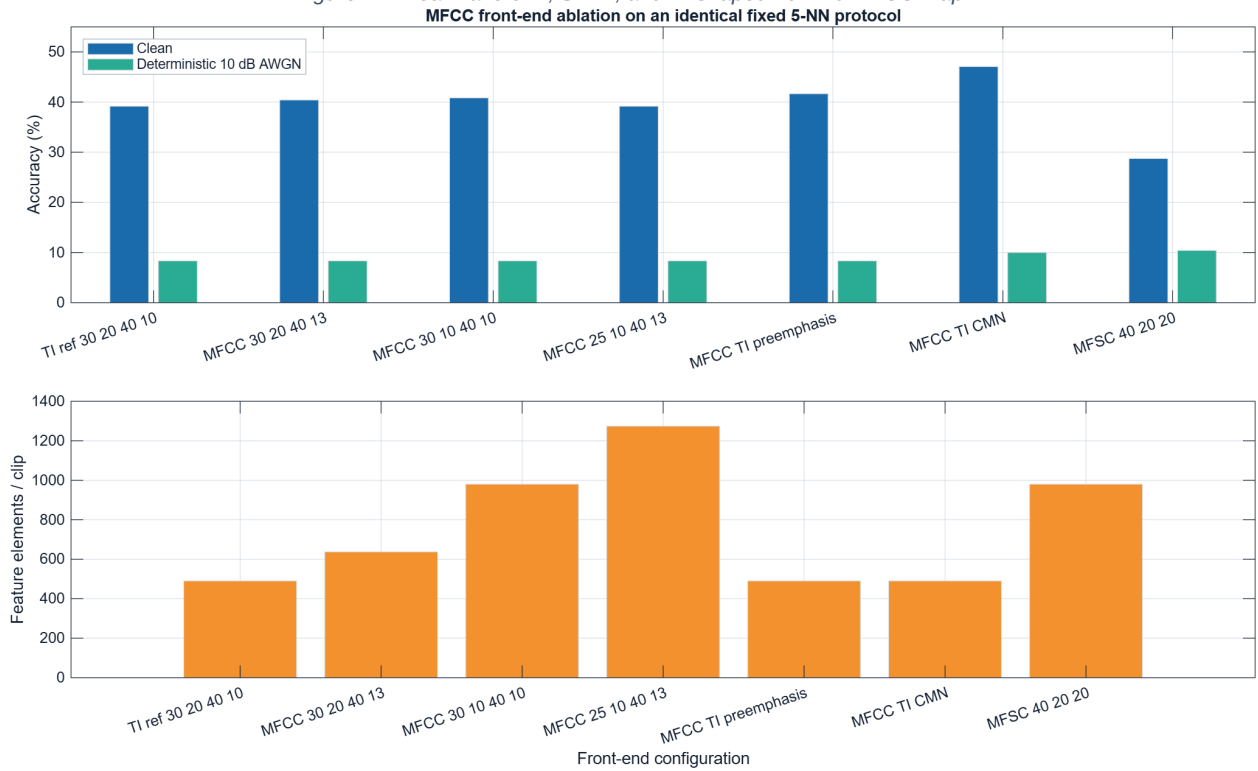


Figure A2. Controlled front-end accuracy and input-size comparison.

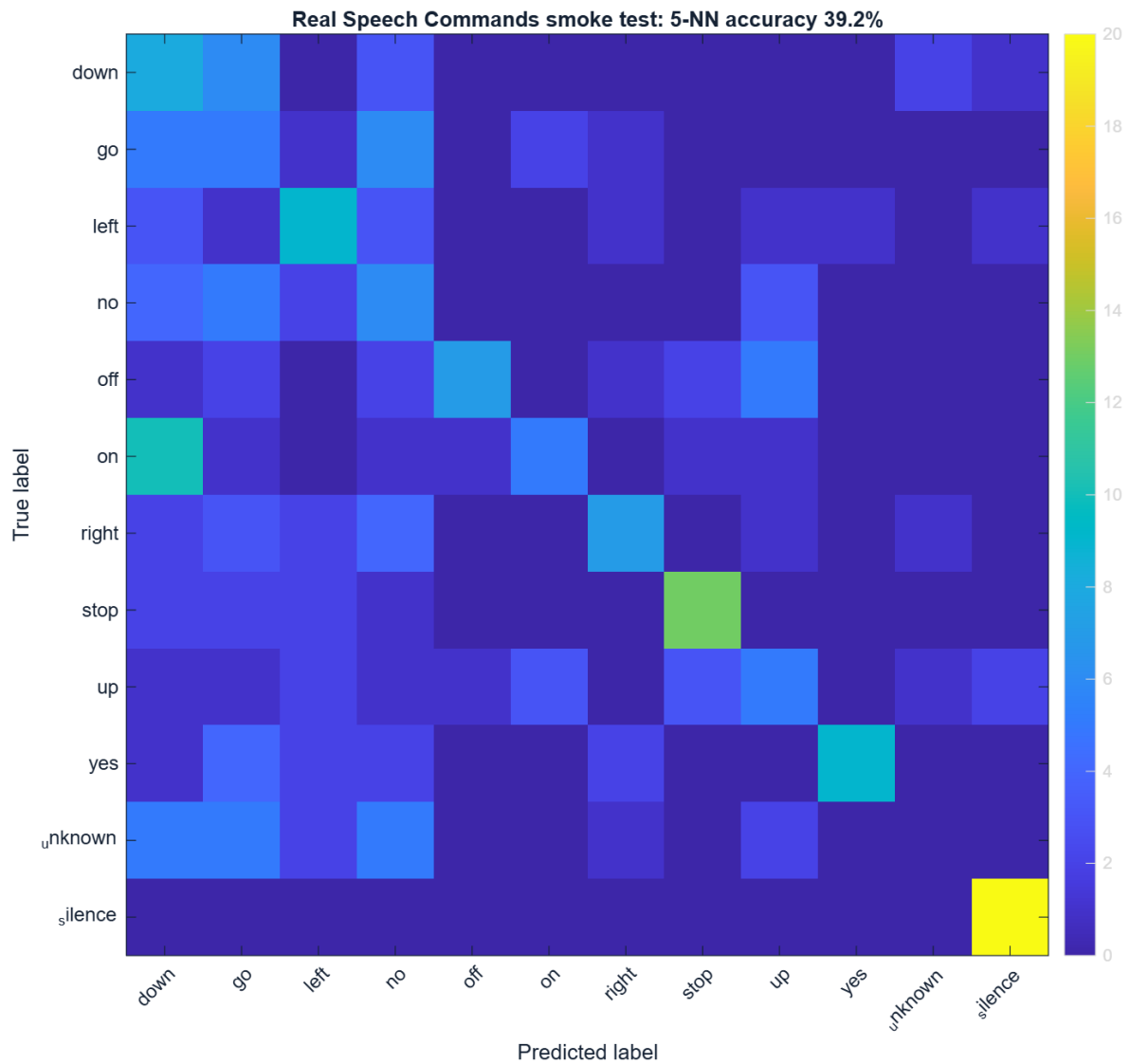


Figure A3. Speaker-disjoint 12-class smoke-test confusion matrix (240 test samples).